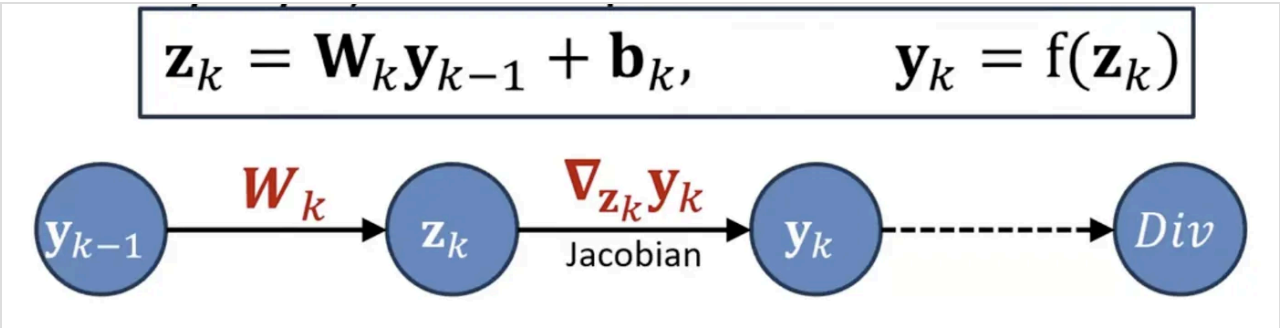
Lec6. Training Part IV

- Gradient Descent
  Training the network

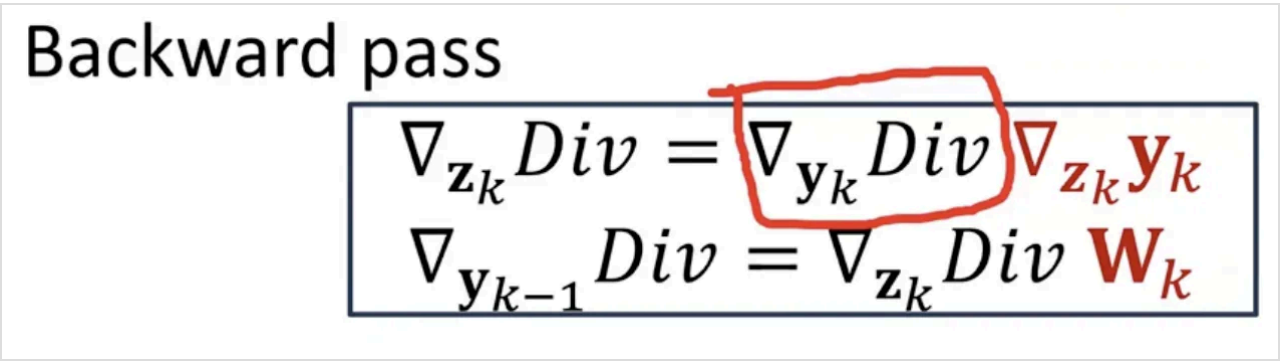
Vector derivatives

- at any layer, **forward pass**



- affine항으로 z값을 구하고 → activation function f(z)를 구하면 됨.
  이때 화살표 위에는, 미분값을 표시하게 되는데
  affine항의 경우, linear하게 W_k가 되고,
  activation은 **jacobian**이 됨.

Backward pass

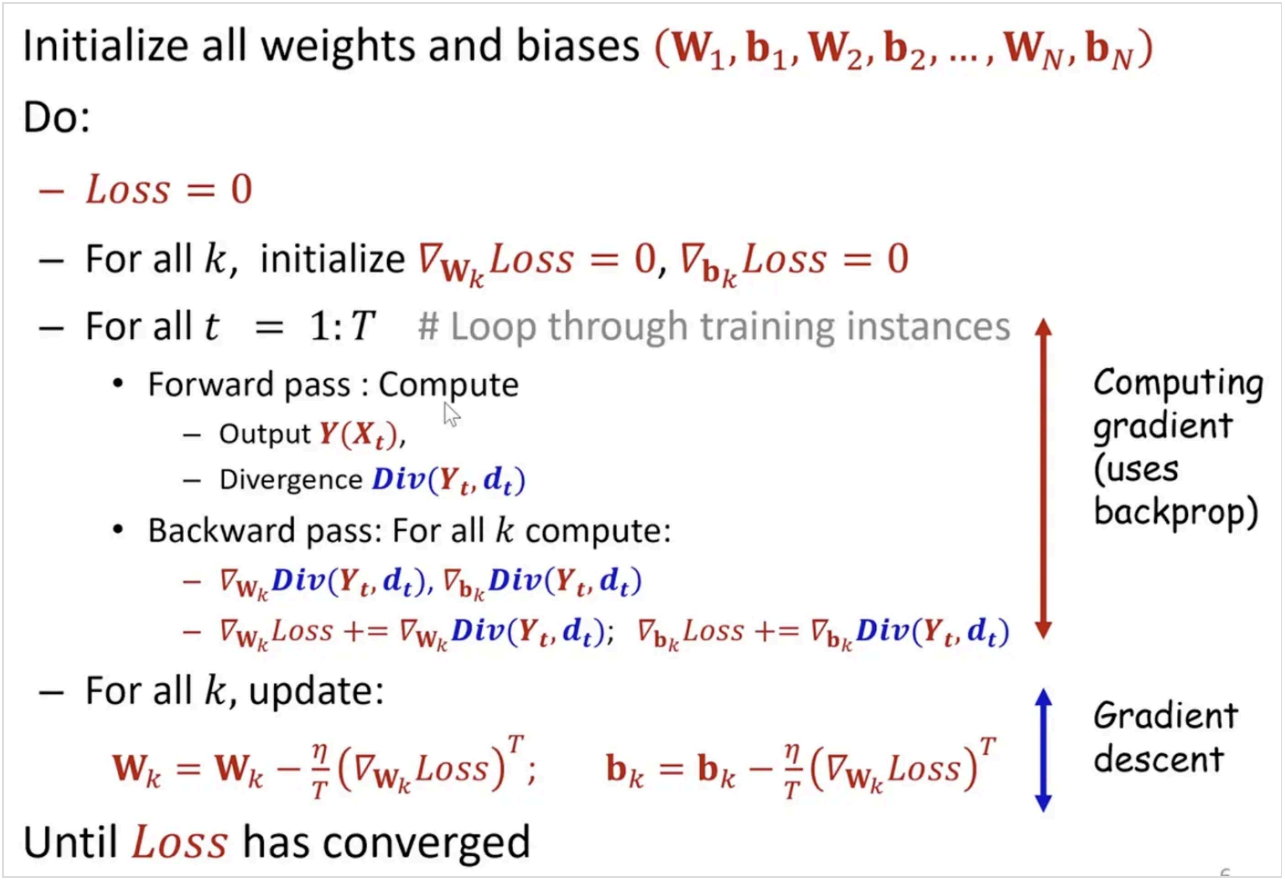


- Forward pass에서는 네트워크 출력과 정답을 이용해 loss Div를 계산한다. Backward pass에서는 chain rule을 이용해 loss의 미분값을 역방향으로 전달한다. 즉, 각 연산의 local derivative 또는 Jacobian을 적절한 순서로 곱하여 이전 계층의 gradient를 구한다.



- Backpropagation은 loss를 각 중간 활성화값에 대해 미분한 결과를 역방향으로 전파한다. 이때 각 계층의 가중치 W_k와 bias b_k에 대한 gradient도 chain rule로 계산한다. 학습에서는 이 파라미터 gradient를 사용해 W_k, b_k를 갱신하며, 입력 자체에 대한 gradient와는 구별해야 한다.

Neural network training algorithm

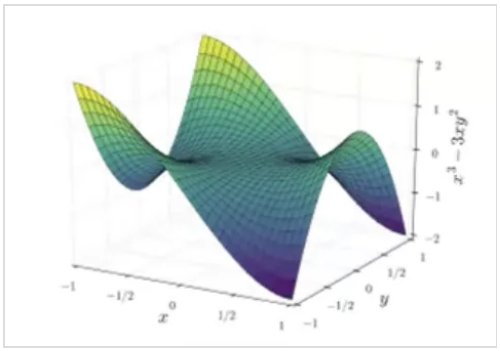
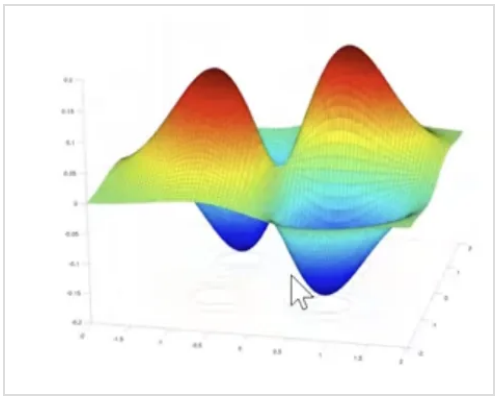
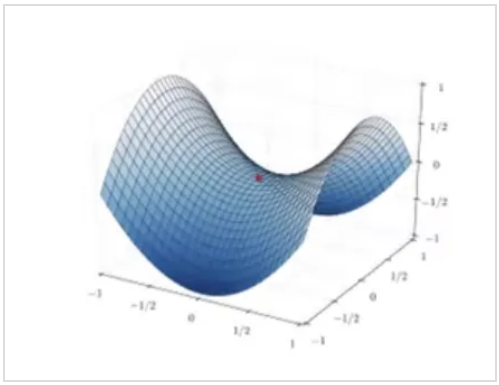


- 정정: 위 강의 캡처의 bias 갱신식에 표시된 ∇_{W_k} Loss는 ∇_{b_k} Loss의 오기입니다. 가중치와 bias는 각각 해당 파라미터에 대한 gradient로 갱신해야 하므로, 올바른 갱신식은 $W_k \leftarrow W_k - (\eta/T)(\nabla_{W_k} \text{Loss})^T$, $b_k \leftarrow b_k - (\eta/T)(\nabla_{b_k} \text{Loss})^T$입니다.

Is backprop always right?

- → loss function의 최소값을 찾는 게 항상, 최적의 분류를 만들지는 않음.

Loss Surface

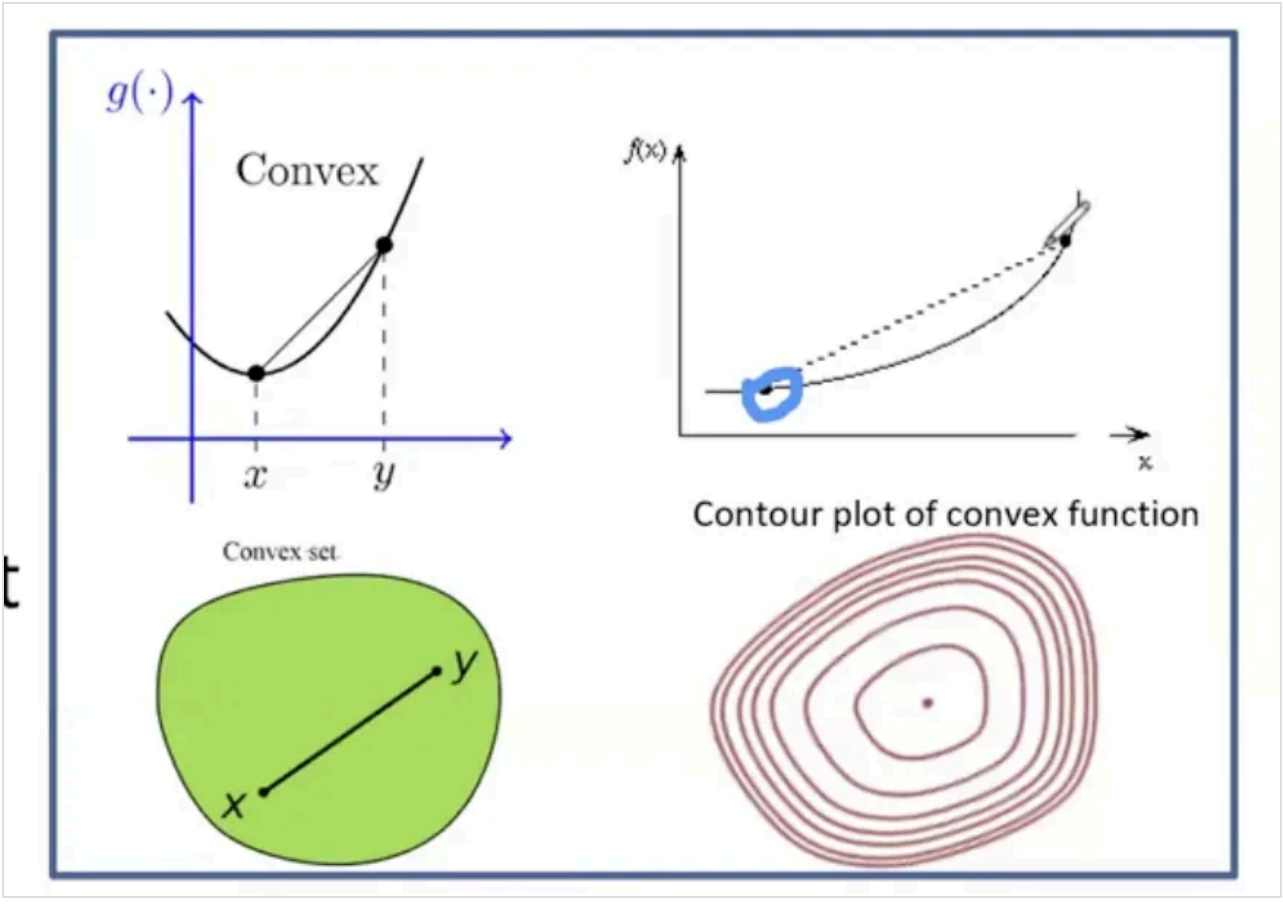




- popular hypothesis
  - in large networks
    → **saddle points** are far more common than local minima

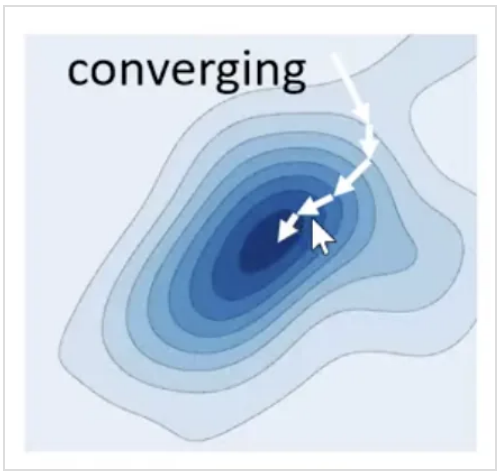
---

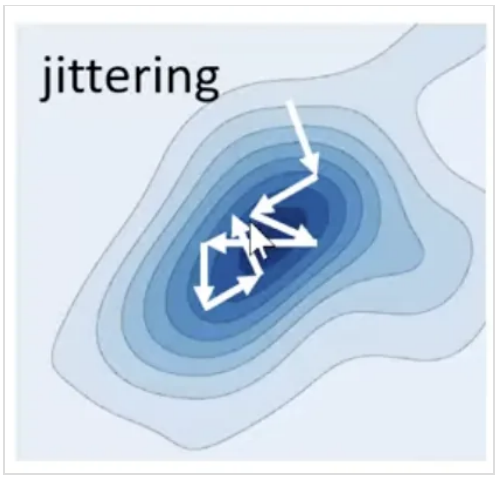
Convex optimization

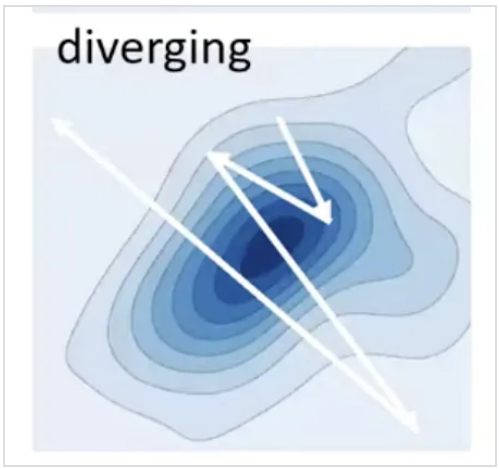
- Convex optimization은 convex한 목적함수를 convex한 실행 가능 영역에서 최소화하는 최적화 문제이다. 함수 f가 convex하다는 것은 정의역의 임의의 x, y와 $0 \leq t \leq 1$에 대해 $f(tx+(1-t)y) \leq tf(x)+(1-t)f(y)$가 성립한다는 뜻이다. Convex optimization에서는 모든 국소 최소점이 전역 최소점이므로 비볼록 최적화에 비해 해의 전역 최적성을 판단하기 쉽다. 다만 일반적인 신경망의 손실함수는 파라미터에 대해 convex하지 않다.



- Convex 함수는 국소 최소점이 모두 전역 최소점이라는 장점이 있다. 하지만 gradient descent의 실제 수렴 속도는 함수의 곡률, smoothness, condition number 및 learning rate에 영향을 받는다. 따라서 convexity는 전역 최적해에 관한 구조적 이점을 제공하지만, 모든 convex 문제가 빠르게 풀린다는 뜻은 아니다.







- convex할 때, gradient descent를 하면.
- converging할 수도, iterating할 수도, diverging할 수도 있음

Convergence for quadratic surfaces

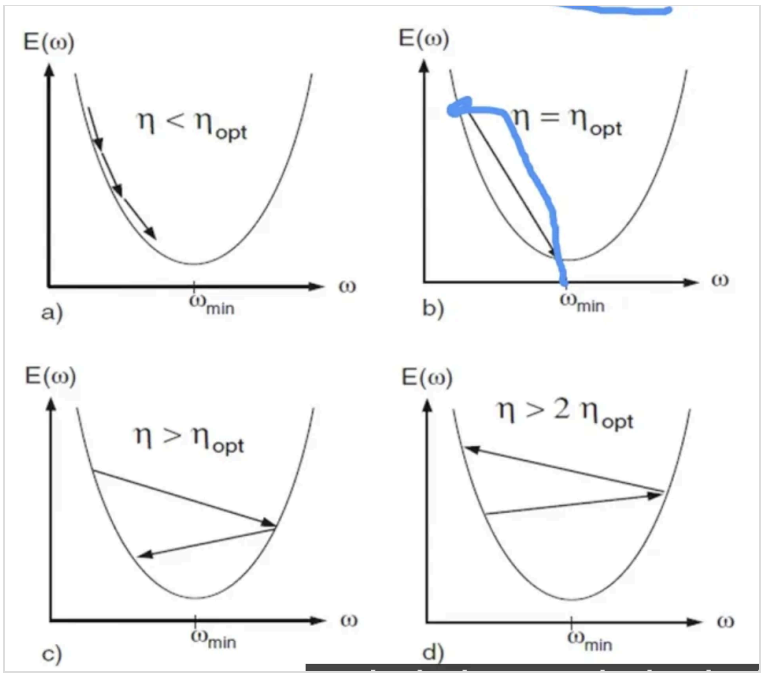
- $MinimizeE = \frac{1}{2}aw^2 + bw + c$
- → 아래와 같이 학습하면 됨.

$$\mathrm{w}^{(k+1)} = \mathrm{w}^{(k)} - \eta \frac{dE\left(\mathrm{w}^{(k)}\right)}{d\mathrm{w}}$$

- 최적의 **step size**는?
- 테일러 전개에 의하면…
- $f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2 + \ldots$
- 이차함수 $E(w)=(1/2)aw^2+bw+c$를 현재 위치 w_k에서 Taylor 전개하면 3차 이상 도함수가 모두 0이므로 다음 식을 정확하게 얻는다. $E(w)=E(w_k)+E'(w_k)(w-w_k)+(1/2)E''(w_k)(w-w_k)^2$. 따라서 이 식은 단순한 2차 근사가 아니라 이차함수에 대한 정확한 Taylor 전개이다.
- $E = E(w_k) + E'(w_k)(w - w_k) + \frac{1}{2}E''(w - w_k)^2$

- 이차함수 $E(w)=(1/2)aw^2+bw+c$에서 $a>0$이라고 가정하면 $E''(w)=a>0$이므로 함수는 strictly convex하며 유일한 최소점을 갖는다. 이 최소점은 $E'(w^*)=0$에서 구할 수 있다. $w^* = -b/a = w_k - (1/a)E'(w_k)$. Gradient descent가 $w_{k+1}=w_k-\eta E'(w_k)$로 갱신되므로, $\eta_{opt}=1/a=1/E''$를 선택하면 현재 위치와 관계없이 한 번의 업데이트로 최소점에 도달한다. 이 결론은 $a>0$인 1변수 이차함수에서 성립한다.
- $(E'')^{-1}E' + (w - w_k) = 0$
  $\Rightarrow w^* = w_k - (E'')^{-1}E'$
- $\Rightarrow \eta = (E'')^{-1}$ 일 때 → 1번만에 최적점으로 이동할 수 있음.
  $\Rightarrow$ **optimal step size**

With non-optimal step size



For $\eta < \eta_{opt}$ the algorithm will converge monotonically

For $2\eta_{opt} > \eta > \eta_{opt}$ we have oscillating convergence

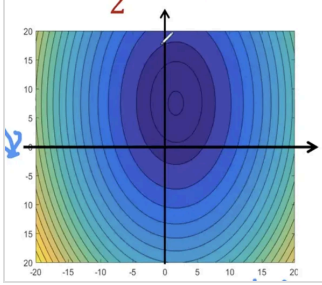
For $\eta > 2\eta_{opt}$ we get divergence

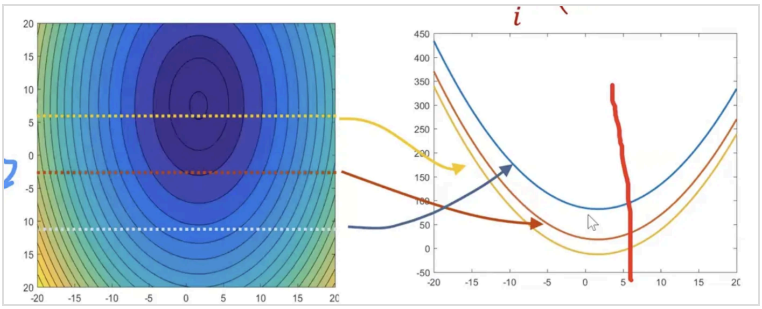
- $a>0$인 1변수 이차함수에서 gradient descent의 수렴 양상은 $\eta_{opt}=1/a$를 기준으로 구분된다. $0<\eta<\eta_{opt}$이면 최소점을 향해 단조 수렴하고, $\eta=\eta_{opt}$이면 한 번에 최소점에 도달한다. $\eta_{opt}<\eta<2\eta_{opt}$이면 최소점을 중심으로 진동하면서 수렴한다. $\eta=2\eta_{opt}$에서는 최소점이 아닌 곳에서 시작할 경우 진폭이 줄어들지 않아 수렴하지 않으며, $\eta>2\eta_{opt}$에서는 일반적으로 발산한다.

---

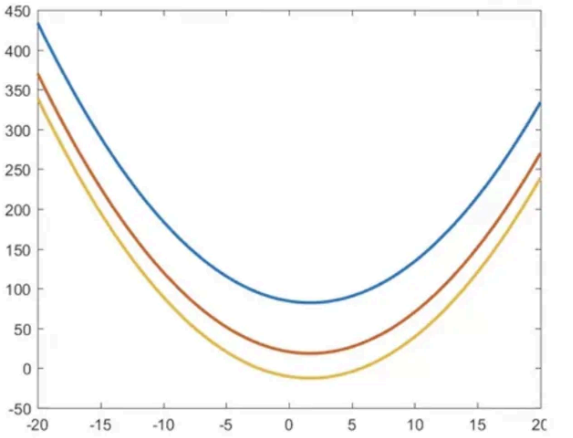
function of multivariate inputs

$$E = \frac{1}{2}\mathbf{w}^T\mathbf{A}\mathbf{w} + \mathbf{w}^T\mathbf{b} + c = \sum_i \left(\frac{1}{2}a_{ii}w_i^2 + b_i w_i\right) + c$$



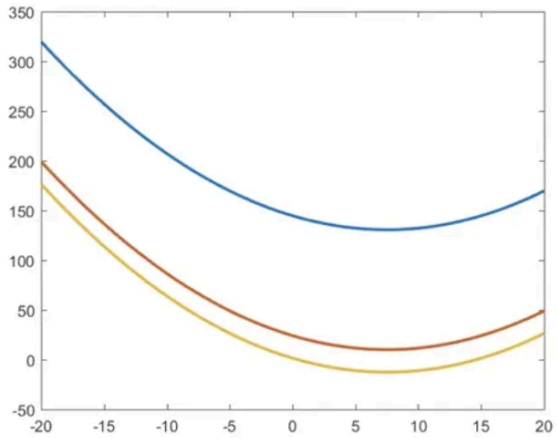


- 다변수 이차함수 $E(w)=(1/2)w^T A w + w^T b + c$에서 다른 좌표를 고정하면 특정 좌표 w_i에 대한 1변수 함수를 얻을 수 있다. 그러나 좌표별 독립성은 일반적으로 성립하지 않는다. A가 대각행렬이고 $a_{ii}>0$이면 교차항이 없어 좌표별로 독립적인 이차함수로 분리되며, 각 방향의 최적 학습률은 $\eta_{i,opt}=1/a_{ii}$이다. 반대로 A에 0이 아닌 비대각 원소가 있으면 좌표들이 결합되어 있으므로 각각을 독립적인 최적화 문제로 취급할 수 없다.



$$E = \frac{1}{2}a_{11}w_1^2 + b_1 w_1 + c + C(\neg w_1)$$

$$\eta_{1,opt} = a_{11}^{-1}$$



$$E = \frac{1}{2}a_{22}w_2^2 + b_2 w_2 + c + C(\neg w_2)$$

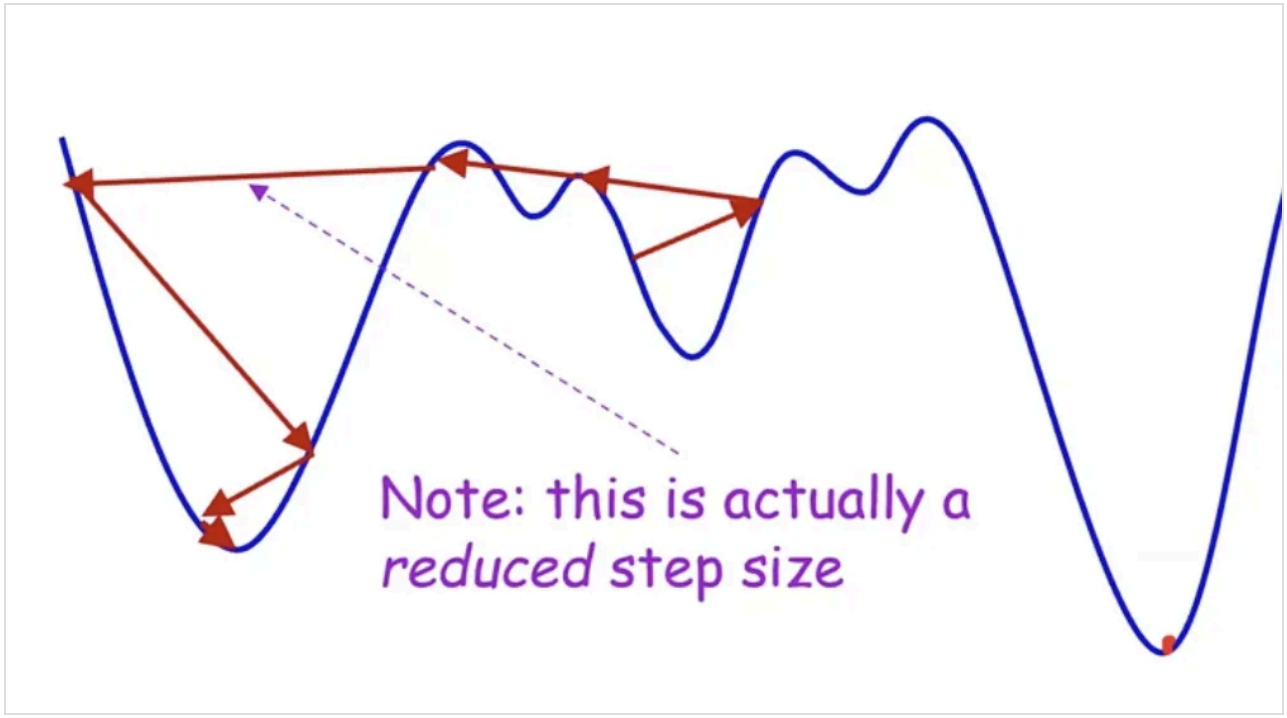
$$\eta_{2,opt} = a_{22}^{-1}$$

- ⇒ 서로 다른 **optimal step size**를 가지게 됨.
- 다변수 이차함수 $E(w)=(1/2)w^T A w + w^T b + c$에서 A가 대칭 양정치 행렬이라고 가정하면, 고정 학습률을 사용하는 gradient descent가 모든 초기점에서 유일한 최소점으로 수렴하기 위한 필요충분조건은 다음과 같다. $0 < \eta < 2/\lambda_{max}(A)$ 여기서 $\lambda_{max}(A)$는 Hessian A의 최대 고유값이다. A가 대각행렬이면 $\lambda_{max}(A)=\max_i a_{ii}$이므로 이 조건은 $0 < \eta < 2 \min_i \eta_{i,opt}$, $\eta_{i,opt}=1/a_{ii}$ 로 표현할 수 있다. 즉, 여러 방향에 하나의 학습률을 적용할 때는 곡률이 가장 큰 방향에서도 발산하지 않도록 학습률을 제한해야 한다. 다만 이 조건은 수렴을 보장하는 범위이지, 수렴 속도가 가장 빠른 학습률을 직접 알려 주는 식은 아니다.

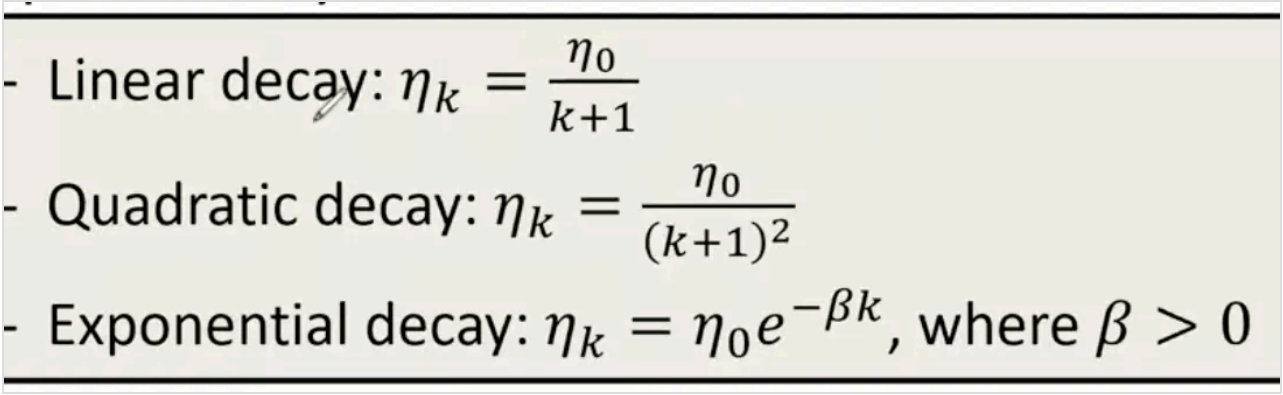
$$\eta < 2 \min_i \eta_{i,opt}$$

- → 충분히 커서 느리지 않으면서, divergence않을만큼 작은 step size를 찾아야 함.

Decaying learning rate



- Learning rate decay는 학습이 진행됨에 따라 step size를 감소시키는 방법이다. 학습 초기에는 비교적 큰 학습률을 사용해 빠르게 이동하고, 후반에는 학습률을 줄여 최적점 주변의 진동이나 지나치게 큰 업데이트를 완화할 수 있다. 대표적인 schedule: Inverse-time decay $\eta_k = \eta_0/(k+1)$, Inverse-quadratic decay $\eta_k = \eta_0/(k+1)^2$, Exponential decay $\eta_k = \eta_0 \cdot \exp(-\beta k)$ ($\beta>0$). η_0는 초기 학습률, k는 업데이트 횟수, β는 지수 감소 상수이다. 학습률을 감소시키면 후반부 진동을 완화할 수 있지만, 줄인다는 사실만으로 수렴이 보장되지는 않는다. 지나치게 빠르게 감소하면 도달 전에 이동량이 매우 작아질 수 있다. 초기 학습률과 감소 속도를 함께 선택해야 하며, 지나치게 빠른 decay가 학습을 방해할 수 있다.

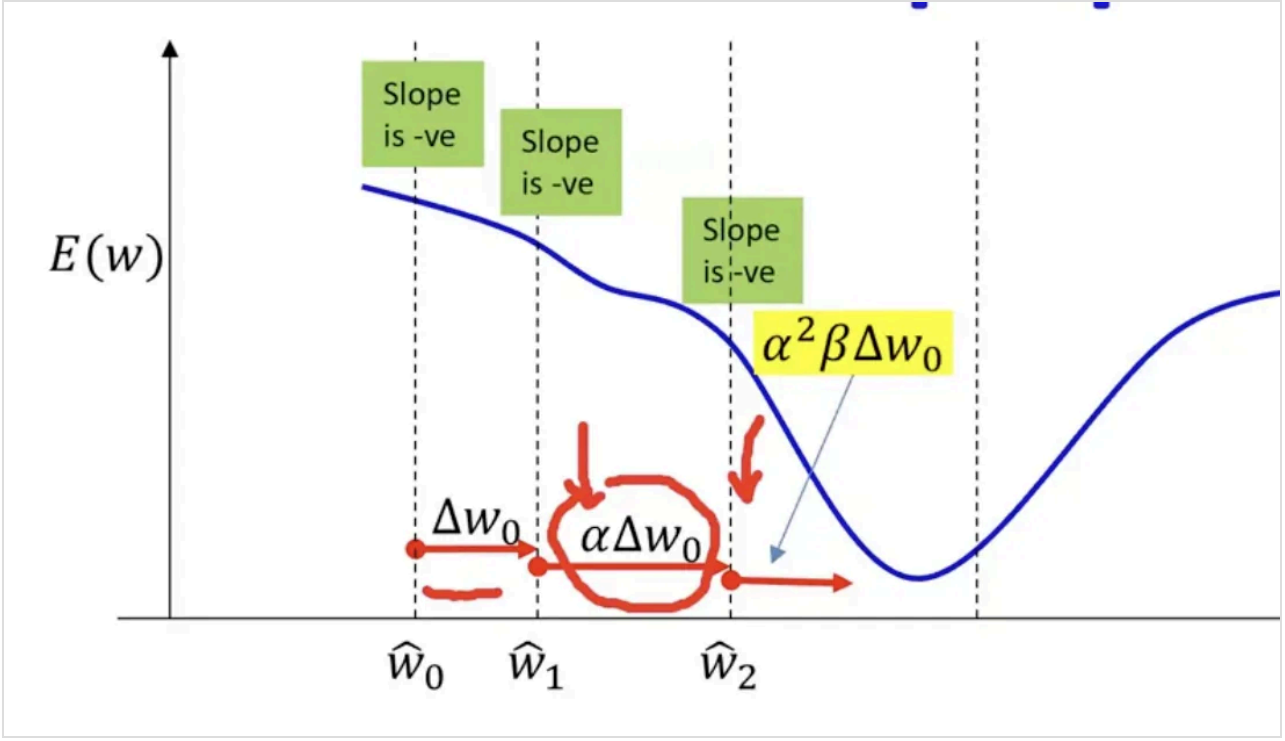


- 강의에서 linear decay라고 표시한 $\eta_k = \eta_0/(k+1)$은 분모가 k의 일차식인 역시간형(inverse-time) 감소 방식이다. Quadratic decay라고 표시한 $\eta_k = \eta_0/(k+1)^2$은 역제곱형(inverse-quadratic) 감소 방식이다. 두 식은 모두 학습률이 k의 증가에 따라 감소하지만, 학습률 자체가 k에 대해 선형 또는 이차적으로 변화한다는 의미는 아니다. 특히 일반적인 linear decay라는 표현은 $\eta_k = \max(0, \eta_0 - ck)$처럼 일정한 양만큼 감소시키는 방식에도 사용되므로 문맥을 구별해야 한다.

Derivative-inspired algorithms

Rprop

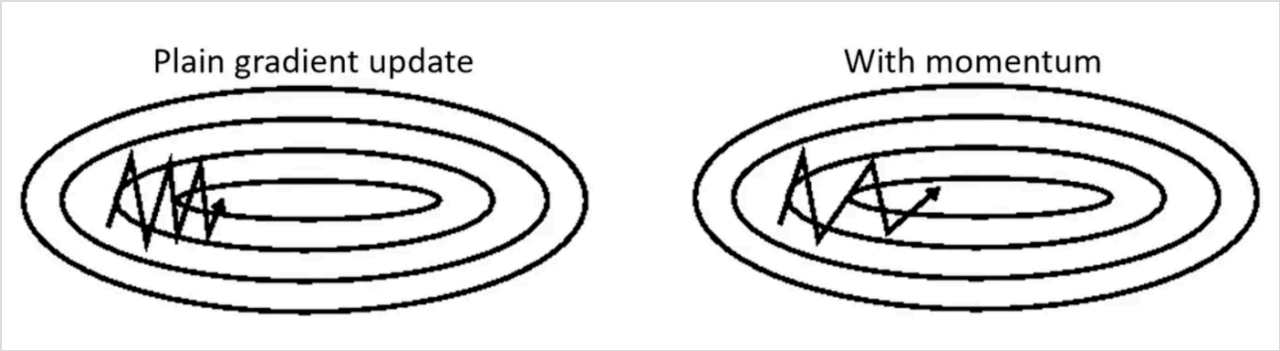
- Rprop(Resilient Backpropagation)은 각 파라미터마다 별도의 step size를 관리하고 gradient의 부호를 이용하여 이동 방향을 결정하는 최적화 알고리즘이다. 현재 gradient를 $g_i(k)$, 이전 gradient를 $g_i(k-1)$, 현재 step size를 $\Delta_i(k)$라고 하자. 두 gradient의 곱이 양수이면($g_i(k) \cdot g_i(k-1) > 0$) 부호가 유지되었으므로 step size를 증가시킨다: $\Delta_i(k) = \min(\alpha \cdot \Delta_i(k-1), \Delta_{max})$, $\alpha > 1$. 곱이 음수이면 부호가 반전되었으므로 감소시킨다: $\Delta_i(k) = \max(\beta \cdot \Delta_i(k-1), \Delta_{min})$, $0 < \beta < 1$. 기본 이동 방향은 gradient 부호의 반대 방향이며, 별도 처리가 없는 업데이트에서는 $-\text{sign}(g_i(k)) \cdot \Delta_i(k)$만큼 이동한다. 부호 반전 시 이전 업데이트를 되돌리는 backtracking 적용 여부는 변형에 따라 다르다: Rprop+는 되돌림 사용, Rprop−는 미사용, iRprop+는 손실 증가 시 되돌림. 따라서 Rprop의 핵심은 gradient 부호에 따른 파라미터별 step size의 적응적 조절이지, 모든 부호 반전에서 반드시 이전 위치로 돌아가는 것은 아니다.



- Rprop에서는 gradient의 부호가 연속해서 유지되면 해당 파라미터의 step size가 증가할 수 있다. 이로 인해 특정한 비볼록 손실함수에서는 작은 local minimum 주변을 건너뛰어 다른 영역으로 이동하는 현상이 발생할 수 있다. 그러나 이 이동이 반드시 더 낮은 손실을 갖는 local minimum으로 이어진다는 보장은 없다. Step size가 지나치게 커지면 최적점을 지나치거나 손실이 증가하고, 경우에 따라 발산할 수도 있다. 따라서 local minimum을 건너뛰는 현상은 가능한 결과 중 하나이며, Rprop이 항상 더 좋은 local minimum을 찾아준다는 의미는 아니다.
- Quickprop은 손실함수가 각 파라미터 방향에서 국소적으로 이차함수에 가깝다고 가정하고, 이전과 현재의 gradient를 이용해 2차 미분을 근사하는 최적화 알고리즘이다. 파라미터 w_i의 현재 gradient를 $g_i(k)$, 이전 gradient를 $g_i(k-1)$이라고 하자. 두 위치 사이의 변화량이 0이 아니면 곡률을 $h_i \approx [g_i(k) - g_i(k-1)] / [w_i(k) - w_i(k-1)]$로 근사할 수 있다. Newton형 업데이트는 $\Delta w_i \approx -g_i(k)/h_i$이다. gradient의 변화량으로 곡률을 추정하고, 곡률이 큰 방향과 작은 방향에 서로 다른 크기의 업데이트를 적용한다. Hessian 전체를 직접 계산하지 않고도 2차 미분 정보를 일부 활용할 수 있다. 그러나 곡률 추정값이 매우 작거나 부정확하면 업데이트가 지나치게 커질 수 있어, 실제 Quickprop에는 업데이트 증가율 제한 등 추가 안정화 규칙이 사용된다. 정리하면 Rprop은 gradient의 부호 변화로 step size를 조정하지만, Quickprop은 연속된 gradient의 차이로 국소 곡률을 추정하여 업데이트를 계산한다.

---

momentum



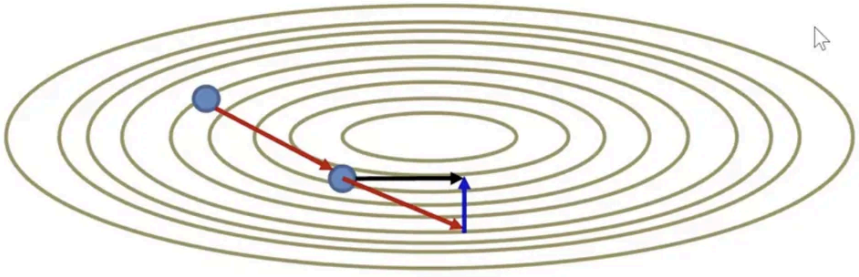
- Momentum은 이전 업데이트 방향을 일정 비율로 유지하면서 현재 gradient를 반영하는 gradient descent의 변형이다. 현재 gradient를 $g_k = \nabla L(w_{(k-1)})$, momentum 계수를 β, learning rate를 η라고 하자. 일반적인 momentum update는 $v_k = \beta \cdot v_{(k-1)} - \eta \cdot g_k$, $w_k = w_{(k-1)} + v_k$로 표현된다. 보통 $0 \leq \beta < 1$, $\eta > 0$을 사용한다. 여기서 v_k는 현재 업데이트 벡터이고, $\beta \cdot v_{(k-1)}$은 이전 업데이트의 영향을 감쇠시켜 유지하는 항이다. Gradient가 연속적으로 비슷한 방향을 가리키면 업데이트가 누적되어 이동이 빨라질 수 있다. 반대로 gradient가 반복적으로 반대 방향으로 바뀌는 방향에서는 이전 업데이트의 영향과 현재 gradient가 일부 상쇄되어 진동이 완화될 수 있다. 따라서 momentum은 길고 좁은 손실 곡면에서 일반 gradient descent의 지그재그 이동을 줄이고 최적화를 빠르게 할 수 있다. 다만 이러한 효과는 목적함수의 곡률, learning rate 및 momentum 계수에 따라 달라지며, momentum이 항상 더 빠른 수렴이나 발산 방지를 보장하는 것은 아니다.
- $x_1, x_2, \ldots$이 있으면
  $\rightarrow \bar{x}_k = \alpha \bar{x}_{k-1} + x_k$

running average는

- Running average는 새로운 값이 들어올 때 과거 정보의 영향을 일부 유지하면서 현재 데이터를 반영하는 방법이다. 대표적인 방법인 exponential moving average(EMA)는 $m_k = \alpha \cdot m_{(k-1)} + (1-\alpha) \cdot x_k$로 정의할 수 있으며, $0 \leq \alpha < 1$이다. 여기서 α는 이전 평균의 유지 비율이고, 1−α는 새로운 데이터의 반영 비율이다. 초기값을 $m_0 = 0$으로 설정하면 초기에 평균이 0 쪽으로 치우칠 수 있으며, 필요에 따라 $m_k/(1-\alpha^k)$와 같은 bias correction을 사용할 수 있다. 반면 강의에 제시된 $s_k = \alpha \cdot s_{(k-1)} + x_k$는 정규화된 평균이 아니라 과거 데이터를 지수적으로 감쇠하여 누적하는 식이다. Momentum은 이러한 감쇠 누적합 구조를 이용해 이전 gradient step의 영향을 유지할 수 있다. 따라서 정규화된 EMA와 momentum에 사용되는 누적합은 유사한 재귀 구조를 가지지만, 현재 입력의 가중치와 값의 규모에서 차이가 있음을 구별해야 한다.

$$gradientstep = -\eta \nabla_W Loss\left(W^{(k-1)}\right)$$
$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} + gradientstep$$
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

Nestorov's Accelerated Gradient



- Nestorov's method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W Loss\left(W^{(k-1)} + \beta \Delta W^{(k-1)}\right)^T$$

$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

- Nesterov Accelerated Gradient(NAG)는 이전 momentum으로 이동할 것으로 예상되는 위치에서 gradient를 먼저 계산하는 방법이다. 일반 momentum은 현재 위치 $w_{(k-1)}$에서 gradient를 계산하지만, Nesterov는 이전 업데이트 벡터를 이용해 look-ahead position을 구한다. Look-ahead position은 $\tilde{w}_k = w_{(k-1)} + \beta \cdot v_{(k-1)}$이다. Nesterov의 업데이트는 $v_k = \beta \cdot v_{(k-1)} - \eta \cdot \nabla L(\tilde{w}_k)$, $w_k = w_{(k-1)} + v_k$로 표현할 수 있다. 즉, 일반 momentum은 현재 위치의 gradient와 이전 이동 방향을 결합하지만, Nesterov는 예상 이동 위치에서 계산한 gradient를 사용한다. 이를 통해 이동하려는 방향의 곡률 정보를 미리 반영하여 업데이트를 조정할 수 있다. 특정 최적화 문제에서는 이러한 방식이 수렴 속도를 개선하지만, 모든 비볼록 신경망에서 일반 momentum보다 빠른 수렴을 보장하는 것은 아니다.